



ΕΡΓΑΣΤΗΡΙΟ 3 – Λίστες

Βασικές Λειτουργίες

Δημιουργία λίστας

- Οι τιμές που θα περιέχονται στη λίστα εσωκλείονται σε αγκύλες [] και χωρίζονται με κόμμα.
- Οι τιμές καταχωρούνται ακολουθιακά, η μια μετά την άλλη.
- Αποτελεί έναν από τους τρόπους δημιουργίας λίστας.
- Μπορούμε να δημιουργήσουμε λίστα και με τη συνάρτηση list()

Παραδείγματα:

```
nums = [3, 5, 8, 13, 21, 34, 55]
users = ['Αργυρώ', 'Γιάννης', 'Κυριάκος']
empty = []
another_empty = list()
```

Πρόσβαση σε στοιχείο της λίστας

- Αποκτούμε πρόσβαση σε ένα στοιχείο της λίστας γράφοντας το όνομα της λίστας και τη θέση του στοιχείου μέσα σε [].
- Η αρίθμηση των θέσεων ξεκινάει πάντα από το 0 και φτάνει μέχρι το πλήθος των στοιχείων της λίστας μειωμένο κατά 1.
- Εναλλακτικά, η αρίθμηση των θέσεων γίνεται κι αντίστροφα, με την τελευταία θέση να αντιστοιχεί στον αριθμό -1.
- Μέσα στις αγκύλες μπορούμε να γράψουμε οποιαδήποτε ακέραια έκφραση.
- Σε περίπτωση που αναφερθούμε σε μια θέση που δεν υπάρχει στη λίστα, δηλαδή σε έναν αριθμό μεγαλύτερο ή ίσο με το πλήθος των στοιχείων της τότε προκύπτει σφάλμα.

Παραδείγματα:

```
nums[3] = 1      # δίνει στο τέταρτο στοιχείο την τιμή 1
nums[-1] = 89    # δίνει στο τελευταίο στοιχείο την τιμή 89
print(nums[0])   # εμφανίζει το πρώτο στοιχείο της λίστας
```

Πλήθος στοιχείων λίστας

Συνάρτηση len()

Λειτουργία: Επιστρέφει το πλήθος των στοιχείων λίστας.

Παράδειγμα:

```
if len(nums) == 0:
    print("Η λίστα είναι κενή")
```

**Άθροισμα στοιχείων λίστας**

Συνάρτηση `sum()`

Λειτουργία: Επιστρέφει το άθροισμα των στοιχείων μιας λίστας.

Παράδειγμα:

```
athroisma = sum(nums)
```

Μέγιστο και ελάχιστο στοιχείο λίστας

Συνάρτηση για εύρεση ελαχίστου `min()`

Λειτουργία: Επιστρέφει το ελάχιστο (μικρότερο) στοιχείο μιας λίστας.

Συνάρτηση για εύρεση μεγίστου `max()`

Λειτουργία: Επιστρέφει το μέγιστο (μεγαλύτερο) στοιχείο μιας λίστας.

Παράδειγμα:

```
elaxistos = min(nums)  
megistos = max(nums)
```

Απαρίθμηση στοιχείων λίστας

Εντολή `for`

Λειτουργία: Η εντολή `for` είναι μια εντολή επανάληψης που διατρέχει τα στοιχεία μιας ακολουθίας τιμών, όπως μια λίστα, με τη σειρά που εμφανίζονται.

- Σε κάθε επανάληψη η τιμή του επόμενου στοιχείου της ακολουθίας ανατίθεται σε μια μεταβλητή απαρίθμησης που χρησιμοποιούμε στην `for`.

Παράδειγμα:

```
for arithmos in nums:  
    print(arithmos)
```

Πράξεις**Τεμαχισμός λίστας**

Λειτουργία: Δημιουργεί μια νέα λίστα που αντιστοιχεί σε ένα «τεμαχισμένο» τμήμα της αρχικής

- Για να τεμαχίσουμε μια λίστα γράφουμε μέσα σε αγκύλες `[ ]` τρεις αριθμούς: την αρχική θέση του τεμαχισμού, τη θέση τερματισμού του τεμαχισμού (που δεν περιλαμβάνεται στο τελικό τμήμα) και ανά πόσα στοιχεία θα περιλαμβάνονται στο τεμαχισμένο τμήμα, ξεκινώντας από την αρχική θέση.
- Αν παραλείψουμε την αρχική θέση, ο τεμαχισμός ξεκινάει από το πρώτο στοιχείο.
- Αν παραλείψουμε την τελική θέση τότε ο τεμαχισμός φτάνει μέχρι το τέλος της λίστας.
- Αν παραλείψουμε το βήμα τότε παίρνει την τιμή `+1`.

Παραδείγματα:



```
nums[1:4]      # δημιουργεί μια νέα λίστα που περιέχει τα στοιχεία στις
θέσεις 1 έως και 3 της αρχικής λίστας
nums[::2]      # ξεκινώντας από την αρχή της λίστας, δημιουργεί μια νέα
λίστα που περιέχει τα στοιχεία της αρχικής που βρίσκονται σε ζυγές θέσεις
nums[::-1]     # δημιουργεί νέα λίστα, αντίστροφη της αρχικής
```

Συνένωση και πολλαπλασιασμός

Τελεστές +, *

Λειτουργία:

Ο τελεστής + (συνένωση) χρησιμοποιείται ανάμεσα σε δύο λίστες και δημιουργεί μια **νέα** λίστα που περιέχει όλα τα στοιχεία των αρχικών.

Ο τελεστής * έχει ως αποτέλεσμα τη δημιουργία μιας **νέας** λίστας που περιέχει πολλές φορές τα στοιχεία της αρχικής.

Παραδείγματα:

```
n = [1, 2, 3] + [4, 5, 6]      # το n θα περιέχει τα [1, 2, 3, 4, 5, 6]
n = [1, 2, 3] * 3              # το n περιέχει τα [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Έλεγχος ύπαρξης τιμής σε λίστα

Τελεστής in

Λειτουργία: Ελέγχει αν η τιμή βρίσκεται στη λίστα και επιστρέφει αναλόγως είτε True ή False.

Παράδειγμα:

```
if 21 in nums:
    print('To βρήκα στη λίστα')
```

Λειτουργίες

Προσθήκη νέου στοιχείου

Μέθοδος append()

Λειτουργία: Προσθέτει ένα νέο στοιχείο (αριθμός, string, λίστα) στο τέλος της λίστας.

Παραδείγματα:

```
nums.append(89)
arithmoi=[1, 2, 3, 4]
arithmoi.append([5, 6, 7])
print(arithmoi) # [1, 2, 3, 4, [5, 6, 7]]
users.append('Μυρσίνη')
```

Μέθοδος insert()



Λειτουργία: Εισάγει ένα νέο στοιχείο σε οποιαδήποτε θέση της λίστας.

- Η θέση και το στοιχείο εισαγωγής δίνονται ως παράμετροι.

Παραδείγματα:

```
users.insert(1, 'Μελίνα') # εισάγει την τιμή 'Μελίνα' στη 2η θέση της λίστας (θέση 1 είναι η δεύτερη θέση, μιας και οι θέσεις ξεκινούν από 0)
```

Αφαίρεση στοιχείου

Μέθοδος `pop()`

Λειτουργία: Αφαιρεί το τελευταίο στοιχείο της λίστας και το επιστρέφει.

Παράδειγμα:

```
lastnum = nums.pop()
```

Μέθοδος `remove()`

Λειτουργία: Αφαιρεί ένα στοιχείο της λίστας.

- Δέχεται ως παράμετρο το στοιχείο που θ' αφαιρεθεί.
- Αν το στοιχείο δεν υπάρχει στη λίστα τότε προκύπτει σφάλμα

Παράδειγμα:

```
nums.remove(13)
```

Δημιουργία αντιγράφου

Μέθοδος `copy()`

Λειτουργία: Επιστρέφει ένα αντίγραφο μιας λίστας.

Παράδειγμα:

```
otherNums = nums.copy()
```

Εύρεση θέσης στοιχείου

Μέθοδος `index()`

Λειτουργία: Αναζητά τη θέση ενός στοιχείου σε μια λίστα.

Παρατηρήσεις:

- Δέχεται ως παράμετρο το στοιχείο της λίστας, για το οποίο αναζητούμε τη θέση του.
- Αν το στοιχείο δεν υπάρχει στη λίστα τότε προκύπτει σφάλμα

Παράδειγμα:

```
pos = nums.index('Γιάννης')
```



Μέτρηση συχνότητας εμφάνισης στοιχείου

Μέθοδος `count()`

- Βρίσκει τη συχνότητα εμφάνισης ενός στοιχείου σε μια λίστα.

Παρατηρήσεις:

- Δέχεται ως παράμετρο το στοιχείο της λίστας, για το οποίο θέλουμε να βρούμε τη συχνότητα εμφάνισής του (πόσες φορές υπάρχει στη λίστα).
- Αν το στοιχείο δεν υπάρχει στη λίστα τότε επιστρέφει 0.

Παράδειγμα:

```
frequency = nums.count(5)
```

Ταξινόμηση στοιχείων

Μέθοδος `sort()`

- Η λίστα στην οποία εφαρμόζεται μεταβάλλεται.

Παράδειγμα:

```
nums.sort()
```

Μετά την εκτέλεση της εντολής `sort()` η λίστα `nums` είναι ταξινομημένη.

Αντιστροφή στοιχείων

Μέθοδος `reverse()`

- Η λίστα στην οποία εφαρμόζεται μεταβάλλεται.
- Αντιστροφή μπορεί να γίνει και με τεμαχισμό: π.χ. `nums[::-1]`

Παράδειγμα:

```
nums.reverse()
```

Μετά την εκτέλεση της εντολής `reverse()` η λίστα `nums` είναι αντεστραμμένη.

Λίστες και Συναρτήσεις



Μια λίστα μπορεί να σταλεί ως παράμετρος μέσα σε μια συνάρτηση. Στην πραγματικότητα η λίστα δεν αντιγράφεται στη συνάρτηση. Οπότε, η συνάρτηση δεν έχει αντίγραφο της λίστας αλλά στην ουσία «βλέπει» την ίδια λίστα που είναι δηλωμένη στο κυρίως πρόγραμμα που καλεί τη συνάρτηση. Έτσι, αν η συνάρτηση αλλάξει τα περιεχόμενα της λίστας τότε οι αλλαγές αυτές είναι ορατές και στο κυρίως πρόγραμμα. Δείτε το πιο κάτω παράδειγμα. Αυτό ισχύει μόνο με τις λίστες (όχι με άλλους τύπους δεδομένων εισόδου που μάθαμε π.χ. αριθμούς, strings).

```
def incrementOdd(lista):
    for i in range(0, len(lista)):
        if lista[i] % 2 != 0:
            lista[i] = lista[i] + 1

alist = [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
blist = alist[1:7:2]
incrementOdd(alist)

print(alist) # [12, 12, 14, 14, 16, 16, 18, 18, 20, 20]
print(blist) # [12, 14, 16]
# όταν κάνουμε append μια λίστα x σε μια άλλη λίστα y, η λίστα x
# προστίθεται σαν ένα ξεχωριστό στοιχείο στο τέλος της λίστας y
blist.append([18, 20])

print(blist) # [12, 14, 16, [18, 20]]
```

Εργαστηριακές Ασκήσεις

Σε κάποια ερωτήματα του εργαστηρίου αυτού δίνεται ένα κομμάτι κώδικα προς αξιολόγηση και μελέτη. Θα πρέπει να περιγράψετε γιατί παίρνουμε κάθε αποτέλεσμα, και όχι απλά να τρέξετε τον κώδικα για να δείτε τι βγάζει.

1. Ορίστε μια μεταβλητή που να έχει τιμή μια λίστα, η οποία περιέχει τα ονόματα (συμβολοσειρές) των ημερών μιας εβδομάδας.
 - a. Τυπώστε τη λίστα.
 - b. Τυπώστε τα περιεχόμενα της λίστας ένα-ένα (ένα όνομα σε κάθε γραμμή)
 - c. Τυπώστε τα περιεχόμενα της λίστας ένα-ένα τυπώνοντας μαζί με κάθε όνομα και τον αριθμό (σειρά) της ημέρας μέσα στην εβδομάδα

Παράδειγμα εκτέλεσης:

```
Ερώτημα (α)
[Δευτέρα, Τρίτη, Τετάρτη, Πέμπτη, Παρασκευή, Σάββατο, Κυριακή]
Ερώτημα (β)
Δευτέρα Τρίτη Τετάρτη Πέμπτη Παρασκευή Σάββατο Κυριακή
Ερώτημα (γ)
Δευτέρα 1
Τρίτη 2
Τετάρτη 3
Πέμπτη 4
```



Παρασκευή 5

Σάββατο 6

Κυριακή 7

2. Ποια θα είναι τα περιεχόμενα της λίστας ονοματα μετά από τις παρακάτω εντολές;
- ```
onomata = ["Thanasis"]
onomata.insert(1, "Anna")
onomata.insert(0, "Swtiris")
onomata.pop(2)
onomata.append("Marios")
```
3. Πως μπορούμε να υπολογίσουμε τον **μέσο όρο** των στοιχείων μιας αριθμητικής λίστας **με χρήση μόνο δύο συναρτήσεων**;
4. Έστω ότι σας δίνεται μια λίστα αριθμών `lista`. Πως μπορείτε με μια έκφραση (χρησιμοποιώντας έτοιμες συναρτήσεις που μάθαμε) να υπολογίσετε και να τυπώσετε το άθροισμα όλων των αριθμών της λίστας εκτός του μικρότερου.

Για παράδειγμα, εάν δοθεί η εξής λίστα:

```
[9, 4, 6, 8, 2, 5]
```

τότε το αποτέλεσμα που θα πρέπει να τυπώσετε θα είναι 32

5. Υλοποιήστε ένα πρόγραμμα που να διαβάζει από το χρήστη ένα αριθμό, να δημιουργεί μια λίστα με τα 12 πρώτα πολλαπλάσια του αριθμού, να αντιστρέφει τη λίστα και στο τέλος να τυπώνει τη λίστα. **ΒΟΗΘΕΙΑ:** για να εισάγετε ένα νέο στοιχείο σε μια υπάρχουσα λίστα μπορείτε να χρησιμοποιήσετε την μέθοδο λίστας `.append(νέο στοιχείο)`.

Παράδειγμα εκτέλεσης: (με κόκκινο χρώμα η είσοδος του χρήστη)

Δώσε ένα αριθμό: **3**

```
[36, 33, 30, 27, 24, 21, 18, 15, 12, 9, 6, 3]
```

- 6.
- Δημιουργήστε ένα αρχείο με όνομα `oisyntartiseismou.py` και υλοποιήστε μια συνάρτηση με όνομα `reverse12()` που να δέχεται σαν τιμή εισόδου έναν αριθμό και θα επιστρέφει μια λίστα με τα πρώτα 12 πολλαπλάσια της τιμής εισόδου με αντίστροφη σειρά.
  - Γράψτε ένα πρόγραμμα (σε νέο αρχείο, π.χ. `Lab08_6.py`) που θα ζητάει ένα ακέραιο αριθμό από το χρήστη, θα καλεί τη συνάρτηση `reverse12` δίνοντας τον αριθμό, και θα τυπώνει το αποτέλεσμα που επιστρέφει η συνάρτηση